

TanStack Query

Four Concepts, Explained

Server state vs client state, the queryKey contract, why fetch doesn't throw, and stale-while-revalidate — with the real CareerHub code, its rendered UI, and a line-by-line walkthrough.

- 1 Server state vs client state**
The four things useQuery gives you for free
- 2 The queryKey contract**
Cache identity — share it right, or watch data collide
- 3 Why fetch does not throw on HTTP errors**
404/500 resolve — the `res.ok` check is what saves you
- 4 Stale-while-revalidate**
Freshness on focus, without the loading flash

1

Server state vs client state

Four things `useQuery` gives you for free — and what the user suffers without them.

Client state is data your app owns: the search box text, which job is selected, the sort order. **Server state** is data that lives on a server and you only borrow: the job listings. It can be stale, shared across screens, slow to arrive, or fail. `useQuery` exists because server state needs four behaviours that `useState` + `useEffect` never give you.

What <code>useQuery</code> gives you free	What the user suffers if you hand-roll it
Loading & error as first-class values <code>isLoading, isError, error</code>	You wire three state variables and reset them in the right order. Miss one path and the user stares at a blank screen — no spinner, no message — when the request hangs or fails.
Request deduplication via the <code>queryKey</code>	Ten components needing jobs fire ten identical requests. The browser hammers the API on every screen, wasting bandwidth and slowing every mount.
Caching (stale-while-revalidate)	A manual <code>useEffect([])</code> re-fetches from scratch each mount, so every navigation flashes a spinner for data the user saw seconds ago.
Refetch on focus / reconnect + retries	None of this happens by hand. Leave the tab for an hour → stale listings; one network blip → a permanent error with no self-recovery.

The code — one hook, three states (`src/app/page.tsx`)

SRC/APP/PAGE.TSX — THE QUERY

```
32  const {
33    data: jobs,
34    isLoading,
35    isError,
36    error,
37    refetch,
38  } = useQuery({
39    queryKey: ["jobs"],
40    queryFn: fetchJobs,
41  });
```

<code>data: jobs</code>	data renamed to <code>jobs</code> . It is undefined until the fetch resolves, then holds the array. You never set it by hand.
<code>isLoading</code>	true while the first fetch is in flight and no cached data exists yet. Drives the skeleton.
<code>isError</code>	true if the <code>queryFn</code> threw. Drives the error panel.
<code>error</code>	The actual <code>Error</code> object that was thrown — its <code>.message</code> is shown to the user.
<code>refetch</code>	A function that re-runs the same query. Wired to the "Try again" button.

<code>queryKey: ["jobs"]</code>	The cache identity (see Section 2). Any component using this key shares this one fetch.
<code>queryFn: fetchJobs</code>	The async function that does the actual HTTP call (see Section 3). TanStack Query calls it for you.

Those five returned values are the “free” behaviours made visible. The render below picks exactly one of three branches from them — the user can only ever be in one state.

1

The three render states

Mutually exclusive — pending, error, or success.

SRC/APP/PAGE.TSX — THE RENDER (CONDENSED)

```
192  {isPending ? (  
193    // PENDING — only the skeleton grid renders.  
195    <JobListSkeleton />  
196  ) : isError ? (  
197    // ERROR — show the real message + a retry button.  
199    <div role="alert">  
207      {error.message}  
211      <button onClick={() => refetch()}>Try again</button>  
216    </div>  
217  ) : (  
218    // SUCCESS — jobs is defined; render filter bar + grid.  
246    <JobList jobs={visibleJobs} ... />  
255  )}
```

isPending ? ...

First branch. While the very first fetch runs, render **only** the skeleton — no empty grid, no “0 jobs”.

: isError ? ...

Second branch. If the fetch threw, show the diagnostic message from `error.message` and a retry control.

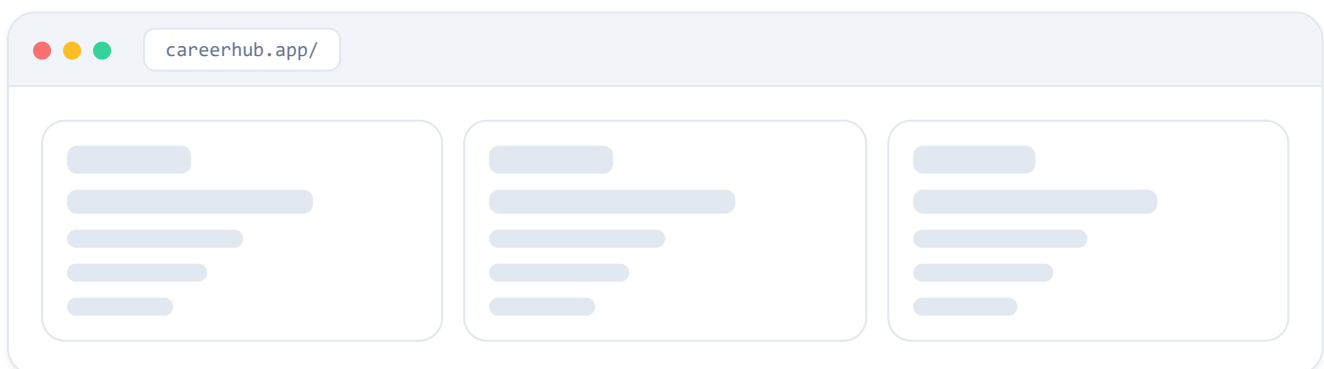
onClick={() => refetch()}

Re-runs the same query. On success the component re-renders straight into the grid — self-recovery, no page reload.

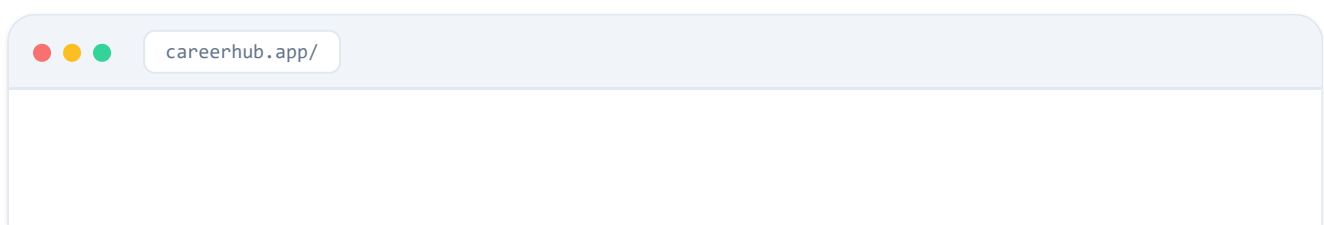
: (...)

Success branch. Reached only when not pending and not errored, so `jobs` is guaranteed defined here.

The UI — what each branch looks like



`isPending === true` — `<JobListSkeleton />` grey bars hold the exact layout the real cards will occupy (no layout shift).

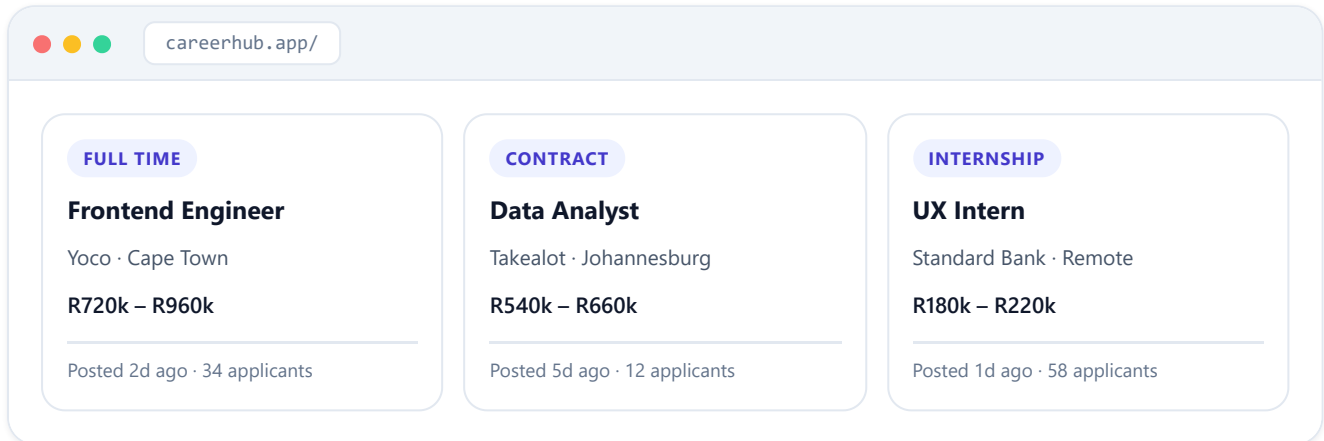


We couldn't load the job listings

Failed to load jobs — server responded with 500 Internal
Server Error

Try again

`isError === true` — `error.message` is shown verbatim; “Try again” calls `refetch()`.



`isPending === false && isError === false` — the success branch renders the live grid from `jobs`.

The payoff

The user is always in exactly one clear state. No blank screen on a hung request, no silent failure — the three booleans guarantee it.

2

The queryKey contract

The cache identity: how results are stored, looked up, deduplicated and invalidated.

The queryKey is an array that acts as a **primary key for the cache**. The rule is simple:

Same key $\Rightarrow$ same data · changed key $\Rightarrow$ a new query and a new fetch

Every component that calls useQuery with ["jobs"] reads from the one cached entry. The first one fetches; the rest get the cached array instantly. That is request deduplication and caching, both keyed off this array.

SRC/APP/PAGE.TSX – THE KEY IN CONTEXT

```
38   } = useQuery({
39     queryKey: ["jobs"],    // ← cache identity for the listings
40     queryFn: fetchJobs,
41   });
```

Two ways to get the key wrong

✗ Accidentally sharing a key

A jobs *list* and a single job *detail* both keyed ["jobs"] overwrite each other — whichever request resolves last wins.

Symptom: the list page shows one job (or crashes on .map), or the detail page renders a whole array of garbage.

RIGHT VS WRONG – SHARING

```
// ✗ WRONG – collision: both write the same cache slot
useQuery({ queryKey: ["jobs"], queryFn: fetchJobs });
useQuery({ queryKey: ["jobs"], queryFn: () => fetchJob(id) });

// ✓ RIGHT – distinct, hierarchical keys
useQuery({ queryKey: ["jobs"], queryFn: fetchJobs });
useQuery({ queryKey: ["jobs", id], queryFn: () => fetchJob(id) });
```

✗ Using a unique key when it should be shared

["jobs", Math.random()] never matches a previous key, so every mount is treated as a brand-new query.

Symptom: a skeleton flashes on every navigation and the cache never warms up — the exact problem TanStack Query was meant to solve.

THE UNSTABLE-KEY TRAP

```
// ✗ WRONG – new key every render, cache can never hit
useQuery({ queryKey: ["jobs", Math.random()], queryFn: fetchJobs });

// ✓ RIGHT – stable key, cache hits across mounts
useQuery({ queryKey: ["jobs"], queryFn: fetchJobs });
```

The mental model

1 · Look up

On mount, TanStack Query hashes the key and checks the cache for an entry.

2 · Hit or fetch

Cached & fresh → return it instantly.
Missing/stale → run queryFn once.

3 · Dedupe

Other components with the same key in flight? They wait on the one request.

Rule of thumb

Put **everything the data depends on** into the key — and nothing that changes randomly. `["jobs"]` for the list, `["jobs", id]` for a detail, `["jobs", { type, sort }]` if the server filters. Stable, descriptive, and as granular as the data.

3

Why fetch does not throw on HTTP errors

A 404 or 500 is a *successful* round-trip — so you must throw yourself.

fetch only **rejects** on a network-level failure: DNS lookup fails, the device is offline, CORS blocks it, or the request is aborted. A **404 or 500 still resolves** — the server was reached and answered, which from fetch's point of view is success. The status lives on `res.ok` (true only for 200–299).

The trap

Remove the `res.ok` check and TanStack Query sees a resolved `queryFn`, so it stores the *error body* as **success data**. The UI leaves loading and renders the success branch against junk — an empty grid, blank fields, or `jobs.map` is not a function crashing the page. The error state never shows.

The code (src/lib/api.ts)

SRC/LIB/API.TS — THE NETWORK LAYER

```
10 export async function fetchJobs(): Promise<JobListing[]> {
16   const baseUrl = process.env.NEXT_PUBLIC_API_URL ?? "";
17   const res = await fetch(`${baseUrl}/api/jobs`);
18
25   if (!res.ok) {
26     throw new Error(`Failed to load jobs - server responded with ${res.status}
    ${res.statusText}`);
27   }
28
29   return (await res.json()) as JobListing[];
30 }
```

<code>: Promise<JobListing[]></code>	The return type is the contract: this resolves to an array of jobs, or it throws. TanStack Query relies on exactly that shape.
<code>baseUrl ... ?? ""</code>	Honour <code>NEXT_PUBLIC_API_URL</code> when set (a real backend); otherwise fall back to <code>""</code> so the relative <code>/api/jobs</code> route is used — instead of fetching undefined/ <code>/api/jobs</code> , which 404s.
<code>await fetch(...)</code>	Fires the request. This line only rejects on a network-level failure — never on a 4xx/5xx.
<code>if (!res.ok)</code>	The critical line. <code>res.ok</code> is false for any non-2xx status. This is the only place an HTTP error becomes a thrown error.
<code>throw new Error(... \${res.status} ...)</code>	Throwing rejects the <code>queryFn</code> promise → TanStack Query flips <code>isError</code> and stores this <code>Error</code> . Including the status makes the message diagnostic, not generic.
<code>return (await res.json())</code>	Reached only on a 2xx. Parse the body and hand the typed array back as data.

With the check

HTTP errors are routed into the `isError` / `error` path → the user sees the red panel with a real message and a retry button (the Section 1 error UI).

Without the check

The error body (e.g. `{"error": "DB down"}`) is treated as *data*. `jobs.map(...)` runs on a non-array → **page crash**, and the carefully-built error UI never appears.

Where the data comes from (`src/app/api/jobs/route.ts`)

SRC/APP/API/JOB/ROUTE.TS – THE MOCK BACKEND

```
21 export async function GET() {  
22   const jobs: JobListing[] = JOBS;  
23   return NextResponse.json(jobs);  
24 }
```

Only GET is handled, so any other method gets Next.js's default 405 — which `fetchJobs` would correctly turn into an `isError` state via the same `res.ok` guard.

4

Stale-while-revalidate

Refresh in the background; keep the old data on screen until the new arrives.

The default `staleTime` is 0, meaning data is considered stale the moment it arrives. When you return to the tab, TanStack Query triggers a **background refetch** — but the data already on screen **stays rendered**. The user sees the existing grid, not a spinner; only the background `isFetching` flag flips, and new data swaps in seamlessly when it lands.

SRC/APP/PROVIDERS.TSX — ONE CLIENT FOR THE WHOLE APP

```
21 export default function Providers({ children }) {
22   const [queryClient] = useState(() => new QueryClient());
23
24   return (
25     <QueryClientProvider client={queryClient}>
26       {children}
27     <ReactQueryDevtools initialIsOpen={false} />
28   </QueryClientProvider>
29 );
30 }
```

`useState(() => new
QueryClient())`

Creates the cache **once per browser session**. The initialiser never re-runs on re-render, and it isn't module-level — so one user's cache can't leak into another's server response.

`QueryClientProvider`

Makes that single client (and its cache) available to every `useQuery` below it. This is what lets `["jobs"]` be shared app-wide.

`ReactQueryDevtools`

Dev-only overlay to watch queries flip between *fresh*, *stale*, and *fetching* — the SWR cycle made visible.

SWR vs the naive approaches

Approach	What the user sees on return to the tab
useQuery (SWR, default)	Existing grid stays on screen. A background refetch runs (<code>isFetching</code> flips); fresh data swaps in with no flash .
<code>useEffect(..., [])</code>	Runs once on mount and never again — never refreshes on focus . Stale data forever until a full reload.
Naive manual fix (clear data, show spinner)	Flashes a loading state over content the user is reading . Jarring, and it throws away good data to show a spinner.

The SWR cycle, step by step

1 · Cached render

2 · Revalidate

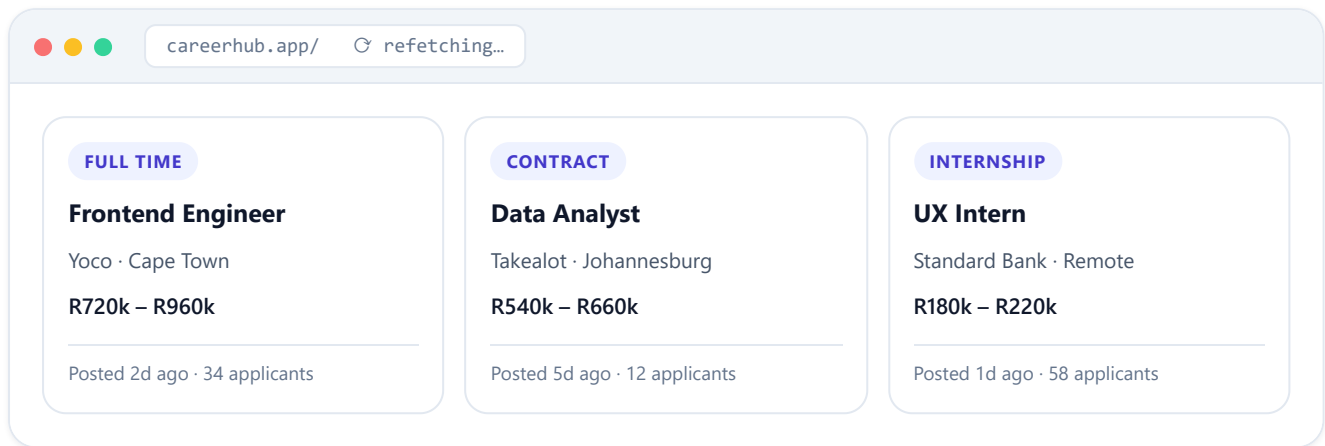
3 · Swap

Return to tab → the previously fetched grid renders immediately from cache.

Because `staleTime: 0`, a background refetch starts. `isFetching` → `true`. UI unchanged.

Fresh data resolves and replaces the old seamlessly. `isFetching` → `false`. No spinner shown.

The UI — stale-while-revalidate vs the naive flash



SWR — *grid stays fully readable while `isFetching === true` revalidates in the background. Contrast with the naive path, which would replace all three cards with the grey skeleton from Section 1.*

The payoff

Freshness without the flash. The user keeps reading current data; the app quietly updates it. That single behaviour — impossible with a one-shot `useEffect` — is the whole reason server state belongs in TanStack Query.